



SSC502 – Laboratório de ICC I

Variáveis Indexadas em C

Prof.Dr. Danilo Spatti

```
tipo_variável nome_variável[tamanho];
```

- Quando o C vê uma declaração como esta ele **reserva** um espaço na **memória** suficientemente **grande** para armazenar o número de **células especificadas** em **tamanho**.

```
float exemplo[20];
```

- O C irá **reservar** $4 \times 20 = 80$ bytes.
- Estes **bytes** são **reservados** de maneira **contígua**.
- Em C, o **indexador** começa sempre em **zero**.
- No exemplo, o **acesso** seria `exemplo[0]`, `exemplo[1]`, ..., `exemplo[19]`.

- Nada **impede** de **acessar** `exemplo[20]`, `exemplo[30]`, `exemplo[103]`, etc.
- Porque o C **não verifica** se o índice que você usou está **dentro** dos **limites** válidos.
- Se o **programador não** tiver **atenção** com os **limites** de validade para os índices ele corre o risco de ter variáveis **sobrescritas** ou de ver o computador **travar**.
- Bugs **terríveis** podem surgir.

```
1  # include <stdio.h>
2  # include <locale.h>
3
4  int main()
5  {
6      setlocale(LC_ALL, "Portuguese");
7      int num[100];
8      int count = 0;
9      int totalnums;
10     do
11     {
12         printf("\nEntre com um número <-999> p/ terminar: ");
13         scanf("%d", &num[count]);
14         count++;
15     } while (num[count-1] != -999);
16     totalnums = count - 1;
17     printf("\n\nOs números que você digitou foram:");
18     for (count = 0; count < totalnums; count++) printf(" %d", num[count]);
19     return 0;
20 }
```

Entre com um número <-999> p/ terminar: 1

Entre com um número <-999> p/ terminar: 2

Entre com um número <-999> p/ terminar: 3

Entre com um número <-999> p/ terminar: -999

Os números que você digitou foram: 1 2 3

Process returned 0 (0x0) execution time : 13.193 s

Press any key to continue.

- No Exemplo 1, o inteiro `count` é **inicializado** em 0.
- O programa pede pela **entrada** de números até que o usuário **entre** com o número (Flag) -999.
- Os números são **armazenados** no vetor `num`.
- A cada número armazenado, o **contador** do **vetor** é **incrementado** para na próxima iteração **escrever** na **próxima** posição do vetor.

- Quando o usuário digita o flag, o programa **abandona** o **primeiro** loop e **armazena** o total de números **gravados**.
- Por fim, todos os números são **impressos**.
- É bom lembrar aqui que **nenhuma** restrição é **feita** quanto a **quantidade** de números digitados.
- Se o usuário digitar **mais** de 100 **números**, o programa **tentará** ler **normalmente**, mas o programa os escreverá em uma **parte não alocada** de **memória**, pois o espaço alocado foi para **somente** 100 inteiros.

```
tipo_variável nome_variável[linhas][colunas];
```

- A forma geral da declaração de uma matriz **bidimensional** é muito **parecida** com a **declaração** de um vetor.
- É muito importante **ressaltar** que, nesta **estrutura**, o índice da **esquerda** indexa as **linhas** e o da **direita** indexa as **colunas**.

- Quando vamos preencher ou ler uma matriz no C o índice mais à direita varia mais rapidamente que o índice à esquerda.
- Mais uma vez é bom lembrar que, na linguagem C, os índices variam de zero ao valor declarado, menos um; mas o C não vai verificar isto para o usuário.
- Manter os índices na faixa permitida é tarefa do programador.

```
1  # include <stdio.h>
2  # include <locale.h>
3
4  int main()
5  {
6      setlocale(LC_ALL, "Portuguese");
7      int mtrx[20][10];
8      int i,j,count;
9      count = 1;
10     for (i = 0; i < 20; i++)
11     {
12         for (j = 0; j < 10; j++)
13         {
14             mtrx[i][j] = count;
15             count++;
16         }
17     }
18     return 0;
19 }
```

```
1  # include <stdio.h>
2  # include <locale.h>
3
4  int main()
5  {
6      setlocale(LC_ALL, "Portuguese");
7      int mtrx[20][10];
8      int i,j,count;
9      count = 1;
10     for (i = 0; i < 20; i++)
11     {
12         for (j = 0; j < 10; j++)
13         {
14             mtrx[i][j] = count;
15             count++;
16         }
17     }
18     for (i = 0; i < 20; i++)
19     {
20         for (j = 0; j < 10; j++)
21         {
22             printf("%d ",mtrx[i][j]);
23         }
24         printf("\n");
25     }
26     return 0;
27 }
```

```
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20
21 22 23 24 25 26 27 28 29 30
31 32 33 34 35 36 37 38 39 40
41 42 43 44 45 46 47 48 49 50
51 52 53 54 55 56 57 58 59 60
61 62 63 64 65 66 67 68 69 70
71 72 73 74 75 76 77 78 79 80
81 82 83 84 85 86 87 88 89 90
91 92 93 94 95 96 97 98 99 100
101 102 103 104 105 106 107 108 109 110
111 112 113 114 115 116 117 118 119 120
121 122 123 124 125 126 127 128 129 130
131 132 133 134 135 136 137 138 139 140
141 142 143 144 145 146 147 148 149 150
151 152 153 154 155 156 157 158 159 160
161 162 163 164 165 166 167 168 169 170
171 172 173 174 175 176 177 178 179 180
181 182 183 184 185 186 187 188 189 190
191 192 193 194 195 196 197 198 199 200
```

```
Process returned 0 (0x0)   execution time : 0.031 s
Press any key to continue.
```

- Um vetor **também** pode ser **inicializado** durante a sua **declaração**.
- Regra geral

```
tipo nome_identificador [tamanho] =  
{lista_de_valores};
```

```
int i[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

- Matrizes podem ser **inicializadas** da **mesma** forma.

```
int mat[10][2] = {{1, 1}, {2, 4}, {3, 9}, {4, 16},  
                 {5, 25}, {6, 36}, {7, 49}, {8, 64},  
                 {9, 81}, {10, 100}};
```

- No C uma **string** é um **vetor** de **caracteres terminado** com um caractere **nulo**.
- O caractere **nulo** é um caractere com valor **inteiro** igual a **zero** (código ASCII igual a 0).
- O terminador nulo **também** pode ser **escrito** usando a convenção de **barra invertida** do C como sendo '**\0**'.

```
char nome_string[tamanho];
```

- A linha de código **anterior declara** um vetor de **caracteres** (uma string) com número de posições igual a **tamanho**.
- Note que, como temos que **reservar** um caractere para ser o terminador **nulo**, temos que **declarar** o **comprimento** da string como sendo, **no mínimo**, um **caractere maior** que a **maior string** que pretendemos **armazenar**.
- Vamos supor que declaremos uma string de **7 posições** e coloquemos a palavra **Joao** nela:

J	o	a	o	\0	...	...
---	---	---	---	----	-----	-----

- No caso **acima**, as **duas células** não usadas têm valores **indeterminados**.
- Isto acontece porque o C não **inicializa variáveis**, cabendo ao **programador** esta tarefa.
- Portanto as únicas células que são **inicializadas** são as que contêm os **caracteres** 'J', 'o', 'a', 'o' e '\0' .

- Se quisermos ler uma **string** fornecida pelo usuário podemos usar a função `gets()`.
- Um exemplo do uso desta função é apresentado no próximo slide.
- A função `gets()` coloca o **terminador** nulo na **string**, quando você aperta a tecla "Enter".

```
1  # include <stdio.h>
2  # include <locale.h>
3
4  int main()
5  {
6      setlocale(LC_ALL, "Portuguese");
7      char string[100];
8      printf("Digite uma string: ");
9      gets(string);
10     printf("\nVocê digitou %s", string);
11     return 0;
12 }
```

Digite uma string: Oh Bella Ciao, Bella Ciao, Bella Ciao!

Você digitou Oh Bella Ciao, Bella Ciao, Bella Ciao!

Process returned 0 (0x0) execution time : 29.599 s

Press any key to continue.

- Neste programa, o tamanho **máximo** da string que você pode entrar é uma **string** de **99 caracteres**.
- Se você entrar com uma string de comprimento **maior**, o programa irá **aceitar**, mas os resultados podem ser **desastrosos**.

- Como as strings são **vetores** de **caracteres**, para se acessar um determinado caractere de uma string, basta **indexarmos** para **acessarmos** o caractere **desejado** dentro da string.
- Suponha uma string chamada str. Podemos acessar a segunda letra de str da seguinte forma:

```
str[1];
```

```
1 # include <stdio.h>
2 # include <locale.h>
3
4 int main()
5 {
6     setlocale(LC_ALL, "Portuguese");
7     char str[10] = "João";
8     printf("\nString: %s", str);
9     printf("\nSegunda letra: %c", str[1]);
10    str[1] = 'U';
11    printf("\nNova segunda letra: %c", str[1]);
12    printf("\nNova String: %s", str);
13    return 0;
14 }
```

```
String: João
Segunda letra: o
Nova segunda letra: U
Nova String: JUão
Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.
```

- Na string do programa anterior, o **terminador nulo** está na posição **4**.
- Das posições 0 a 3, sabemos que temos caracteres **válidos**, portanto **podemos escrevê-los**.

```
1  # include <stdio.h>
2  # include <locale.h>
3
4  int main()
5  {
6      setlocale(LC_ALL, "Portuguese");
7      char letra;
8      printf("\nDigite um caractere: ");
9      letra = getch();
10     printf("\nVocê digitou: %c",letra);
11     printf("\nNúmero ASCII: %d",letra);
12     printf("\nSucessora na tabela ASCII: %c, número: %d",letra+1,letra+1);
13     return 0;
14 }
```

```
Digite um caractere: a
Você digitou: a
Número ASCII: 97
Sucessora na tabela ASCII: b, número: 98
Process returned 0 (0x0)   execution time : 1.419 s
Press any key to continue.
```

- Elabore um programa em C que receba a altura (h) e o sexo de uma pessoa (f – Feminino e m -Masculino) e mostre o seu peso ideal, utilizando as seguintes fórmulas:
 - Masculino – $(72.2 * h) - 58$
 - Feminino – $(62.1 * h) - 44.7$
- Caso o caractere recebido seja diferente de f e m, uma mensagem de caractere inválido deverá ser exibido.

- Faça um programa que receba o estado civil do usuário por meio das seguintes opções:
 - s – (Solteiro)
 - c – (Casado)
 - v – (Viúvo)

- Utilize o switch para exibir na tela qual o estado selecionado pelo usuário.

*****Programa Calculadora*****

Menu de Opções:

* : multiplicação

- : subtração

+ : soma

/ : divisão

Digite dois números e a operação desejada no seguinte formato (2+2):

5*10 _____

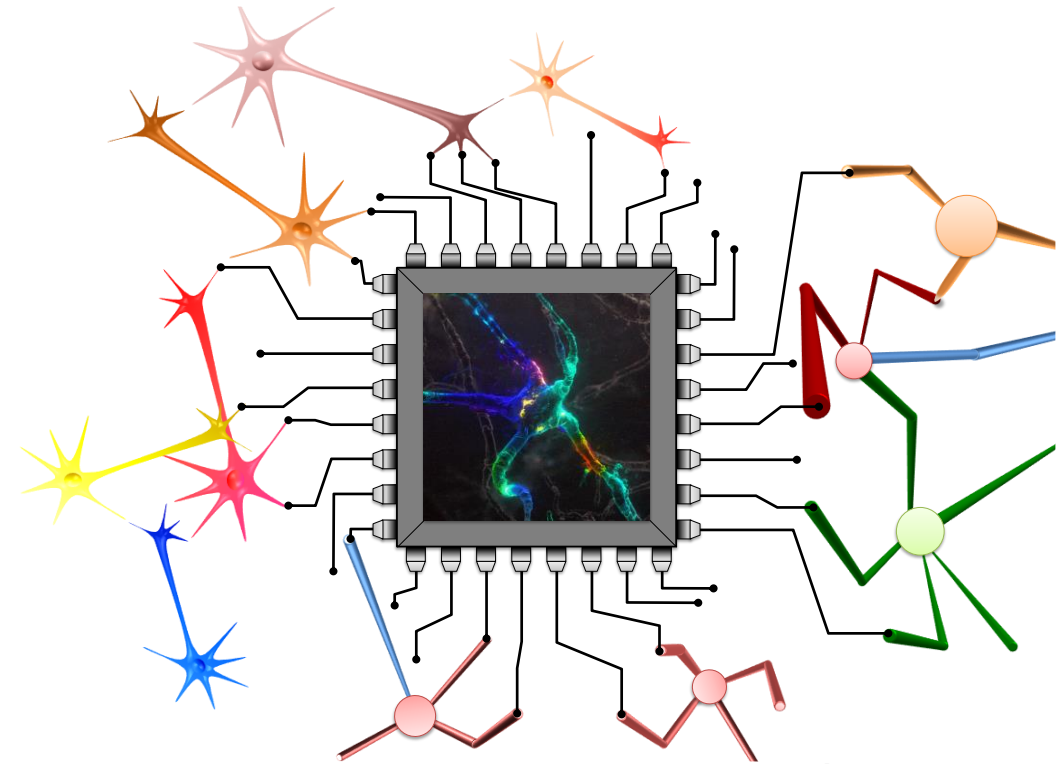
O resultado da operação 5*10: 50

Digitado pelo
usuário

- Faça um programa que inverta uma string:
 - Leia a string com `gets` e armazene-a invertida em outra string.
 - Use o comando `for` para varrer a string até o seu final.

- Faça um programa que inverta uma string:
 - Leia a string com `gets` e armazene-a invertida em outra string.
 - Use o comando `while` para varrer a string até o seu final.

spatti@icmc.usp.br



GE4Bio – Grupo de Estudos em Sinais Biológicos